Table of Contents

5.1. The Run-Time Constant Pool
5.2. Java Virtual Machine Startup
5.3. Creation and Loading
5.3.1. Loading Using the Bootstrap Class Loader
5.3.2. Loading Using a User-defined Class Loader
5.3.3. Creating Array Classes
5.3.4. Loading Constraints
5.3.5. Deriving a Class from a `class` File Representation
5.3.6. Modules and Layers
5.4. Linking
5.4.1. Verification
5.4.2. Preparation
5.4.3. Resolution
5.4.3.1. Class and Interface Resolution
5.4.3.2. Field Resolution
5.4.3.3. Method Resolution
5.4.3.4. Interface Method Resolution
5.4.3.5. Method Type and Method Handle Resolution
5.4.3.6. Dynamically-Computed Constant and Call Site Resolution
5.4.4. Access Control
5.4.5. Method Overriding
5.4.6. Method Selection
5.5. Initialization
5.6. Binding Native Method Implementations
5.7. Java Virtual Machine Termination

Chapter 5. Loading, Linking, and Initializing

The Java Virtual Machine dynamically loads, links and initializes classes and interfaces. Loading is the process of finding the binary representation of a class or interface type with a particular name and *creating* a class or interface from that binary representation. Linking is the process of taking a class or interface and combining it into the run-time state of the Java Virtual Machine so that it can be executed. Initialization of a class or interface consists of executing the class or interface initialization method `<clinit>` (§2.9.2).

In this chapter, §5.1 describes how the Java Virtual Machine derives symbolic references from the binary representation of a class or interface. §5.2 explains how the processes of loading, linking, and initialization are first initiated by the Java Virtual Machine. §5.3 specifies how binary representations of classes and interfaces are loaded by class loaders and how classes and interfaces are created. Linking is described in §5.4. §5.5 details how classes and interfaces are initialized. §5.6 introduces the notion of binding native methods. Finally, §5.7 describes when the Java Virtual Machine terminates.

5.1. The Run-Time Constant Pool

The Java Virtual Machine maintains a run-time constant pool for each class and interface (§2.5.5). This data structure serves many of the purposes of the symbol table of a conventional programming language implementation. The `constant_pool` table in the binary representation of a class or interface (§4.4) is used to construct the run-time constant pool upon class or interface creation (§5.3).

There are two kinds of entry in the run-time constant pool: symbolic references, which may later be resolved (§5.4.3), and static constants, which require no further processing.

The symbolic references in the run-time constant pool are derived from entries in the `constant_pool` table in accordance with the structure of each entry:

- A symbolic reference to a class or interface is derived from a `CONSTANT_Class_info` structure (§4.4.1). Such a reference gives the name of the class or interface in the following form:
  - For a nonarray class or an interface, the name is the binary name (§4.2.1) of the class or interface.
  - For an array class of *n* dimensions, the name begins with *n* occurrences of the ASCII `[` character followed by a representation of the element type:
    - If the element type is a primitive type, it is represented by the corresponding field descriptor (§4.3.2).
    - Otherwise, if the element type is a reference type, it is represented by the ASCII `L` character followed by the binary name of the element type followed by the ASCII `;` character.

  Whenever this chapter refers to the name of a class or interface, the name should be understood to be in the form above. (This is also the form returned by the `Class.getName` method.)
- A symbolic reference to a field of a class or an interface is derived from a `CONSTANT_Fieldref_info` structure (§4.4.2). Such a reference gives the name and descriptor of the field, as well as a symbolic reference to the class or interface in which the field is to be found.
- A symbolic reference to a method of a class is derived from a `CONSTANT_Methodref_info` structure (§4.4.2). Such a reference gives the name and descriptor of the method, as well as a symbolic reference to the class in which the method is to be found.
- A symbolic reference to a method of an interface is derived from a `CONSTANT_InterfaceMethodref_info` structure (§4.4.2). Such a reference gives the name and descriptor of the interface method, as well as a symbolic reference to the interface in which the method is to be found.
- A symbolic reference to a method handle is derived from a `CONSTANT_MethodHandle_info` structure (§4.4.8). Such a reference gives a symbolic reference to a field of a class or interface, or a method of a class, or a method of an interface, depending on the kind of the method handle.
- A symbolic reference to a method type is derived from a `CONSTANT_MethodType_info` structure (§4.4.9). Such a reference gives a method descriptor (§4.3.3).
- A symbolic reference to a *dynamically-computed constant* is derived from a `CONSTANT_Dynamic_info` structure (§4.4.10). Such a reference gives:
  - a symbolic reference to a method handle, which will be invoked to compute the constant's value;
  - a sequence of symbolic references and static constants, which will serve as *static arguments* when the method handle is invoked;
  - an unqualified name and a field descriptor.
- A symbolic reference to a *dynamically-computed call site* is derived from a `CONSTANT_InvokeDynamic_info` structure (§4.4.10). Such a reference gives:
  - a symbolic reference to a method handle, which will be invoked in the course of an *invokedynamic* instruction (§*invokedynamic*) to compute an instance of `java.lang.invoke.CallSite`;
  - a sequence of symbolic references and static constants, which will serve as *static arguments* when the method handle is invoked;
  - an unqualified name and a method descriptor.

The static constants in the run-time constant pool are also derived from entries in the `constant_pool` table in accordance with the structure of each entry:

- A string constant is a `reference` to an instance of class `String`, and is derived from a `CONSTANT_String_info` structure (§4.4.3). To derive a string constant, the Java Virtual Machine examines the sequence of code points given by the `CONSTANT_String_info` structure:

- If the method `String.intern` has previously been invoked on an instance of class `String` containing a sequence of Unicode code points identical to that given by the `CONSTANT_String_info` structure, then the string constant is a `reference` to that same instance of class `String`.
- Otherwise, a new instance of class `String` is created containing the sequence of Unicode code points given by the `CONSTANT_String_info` structure. The string constant is a `reference` to the new instance. Finally, the method `String.intern` is invoked on the new instance.

- Numeric constants are derived from `CONSTANT_Integer_info`, `CONSTANT_Float_info`, `CONSTANT_Long_info`, and `CONSTANT_Double_info` structures (§4.4.4, §4.4.5).

  Note that `CONSTANT_Float_info` structures represent values in IEEE 754 single format and `CONSTANT_Double_info` structures represent values in IEEE 754 double format. The numeric constants derived from these structures must thus be values that can be represented using IEEE 754 single and double formats, respectively.

The remaining structures in the `constant_pool` table - the descriptive structures `CONSTANT_NameAndType_info`, `CONSTANT_Module_info`, and `CONSTANT_Package_info`, and the foundational structure `CONSTANT_Utf8_info` - are only used indirectly when constructing the run-time constant pool. No entries in the run-time constant pool correspond directly to these structures.

Some entries in the run-time constant pool are *loadable*, which means:

- They may be pushed onto the stack by the *ldc* family of instructions (§*ldc*, §*ldc_w*, §*ldc2_w*).
- They may be static arguments to bootstrap methods for dynamically-computed constants and call sites (§5.4.3.6).

An entry in the run-time constant pool is loadable if it is derived from an entry in the `constant_pool` table that is loadable (see Table 4.4-C). Accordingly, the following entries in the run-time constant pool are loadable:

- Symbolic references to classes and interfaces
- Symbolic references to method handles
- Symbolic references to method types
- Symbolic references to dynamically-computed constants
- Static constants

5.2. Java Virtual Machine Startup

The Java Virtual Machine starts up by creating an initial class or interface using the bootstrap class loader (§5.3.1) or a user-defined class loader (§5.3.2). The Java Virtual Machine then links the initial class or interface, initializes it, creates an instance of it (if necessary), and invokes a `main` method as specified in (JLS §12.1.4). The invocation of this method drives all further execution. Execution of the Java Virtual Machine instructions of the instance initialization method (if needed) and `main` method of the initial class or interface may cause linking (and consequently creation) of additional classes and interfaces, as well as invocation of additional methods.

The initial class or interface is specified in an implementation-dependent manner. For example, the initial class or interface could be provided as a command line argument. Alternatively, the implementation of the Java Virtual Machine could itself provide an initial class that sets up a class loader which in turn loads an application. Other choices of the initial class or interface are possible so long as they are consistent with the specification given in the previous paragraph.

In contrast, the selection and invocation of a `main` method of the initial class or interface proceeds according to the implementation-independent rules given in (JLS §12.1.4).

5.3. Creation and Loading

Creation of a class or interface C denoted by the name `N` consists of the construction of an implementation-specific internal representation of C in the method area of the Java Virtual Machine (§2.5.4).

Class or interface creation is triggered by another class or interface D, whose run-time constant pool symbolically references C by means of the name `N` (§5.4.3.1). If `N` does not denote an array class, then the Java Virtual Machine relies on a *class loader* to locate a binary representation for a class or interface called `N` (§4.1). Once a class loader has located a binary representation, it relies in turn on the Java Virtual Machine to derive the class or interface C from the binary representation, and then to create C in the method area. Array classes do not have an external binary representation; they are created by the Java Virtual Machine via a different process.

Class or interface creation may also be triggered by D invoking methods in certain Java SE Platform class libraries (§2.12) such as reflection.

There are two kinds of class loaders: the bootstrap class loader supplied by the Java Virtual Machine, and user-defined class loaders. Every user-defined class loader is an instance of a subclass of the abstract class `ClassLoader`. Applications employ user-defined class loaders in order to extend the manner in which the Java Virtual Machine dynamically creates classes. User-defined class loaders can be used to create classes that originate from user-defined sources. For example, a class could be downloaded across a network, generated on the fly, or extracted from an encrypted file.

When the Java Virtual Machine asks a class loader `L` to locate a binary representation for a class or interface called `N`, `L` *loads* the class or interface C denoted by `N`. `L` may load C directly, by locating a binary representation and asking the Java Virtual Machine to derive and create C from the binary representation. Alternatively, `L` may load C indirectly, by delegating to another class loader which loads C directly or indirectly.

If `L` loads C directly, we say that `L` *defines* C or, equivalently, that `L` is the *defining loader* of C.

Whether `L` loads C directly or indirectly, we say that `L` *initiates* loading of C, or, equivalently, that `L` is an *initiating loader* of C.

Due to class loader delegation, the loader L_1 that initiates loading at the Java Virtual Machine's request may not be the same as the loader L_2 that completes loading by defining the class or interface. In this case, we say that each of L_1 and L_2 *initiates* loading of C, or, equivalently, that each of L_1 and L_2 is an *initiating loader* of C. Any loaders in a delegation chain between L_1 and L_2 are not considered to be initiating loaders of C.

We will sometimes represent a class or interface using the following notation, instead of using an identifier like C or D:

- $\langle N, L_d \rangle$ - where `N` denotes the name of the class or interface and L_d denotes the defining loader of the class or interface.

- N^{L_i} - where N denotes the name of the class or interface and L_i denotes an initiating loader of the class or interface.

It should be clear that loading a class or interface is a joint effort between the Java Virtual Machine and a class loader (or multiple class loaders, if delegation occurs). The ultimate outcome of loading is that the Java Virtual Machine creates a class or interface in its method area, so it is often convenient to say that a class or interface is loaded and thereby created.

The complex back-and-forth nature of loading, combined with the ability of user-defined class loaders to exhibit arbitrary behavior, means that exceptions can be thrown after the Java Virtual Machine has created a class or interface but before every class loader participating in loading has completed. This specification accounts for such exceptions in what is often referred to the process of loading and creating a class or interface.

The Java Virtual Machine uses one of three procedures to create a class or interface C denoted by the name N in the run-time constant pool of a class or interface D:

- If N denotes either a nonarray class or an interface, and D was defined by the bootstrap class loader, then the bootstrap class loader initiates loading of C (§5.3.1).
- If N denotes either a nonarray class or an interface, and D was defined by a user-defined class loader, then that same user-defined class loader initiates loading of C (§5.3.2).
- If N denotes an array class, then the Java Virtual Machine creates an array class C denoted by N, in association with the defining loader of D (§5.3.3).

  Although the defining loader of D is relevant in the course of creating an array class, it is not used to load and thereby create the array class.

If an error occurs during loading of a class or interface - either when a class loader is locating a binary representation, or when the Java Virtual Machine is deriving and creating a class from it - then the error must be thrown at a point in the program that (directly or indirectly) uses the class or interface being loaded.

A well-behaved class loader should maintain three properties:

- *Given the same name, a good class loader should always return the same `Class` object.*
- *If a class loader L_1 delegates loading of a class C to another loader L_2, then for any type T that occurs as the direct superclass or a direct superinterface of C, or as the type of a field in C, or as the type of a formal parameter of a method or constructor in C, or as a return type of a method in C, L_1 and L_2 should return the same `Class` object.*
- *If a user-defined classloader prefetches binary representations of classes and interfaces, or loads a group of related classes together, then it must reflect loading errors only at points in the program where they could have arisen without prefetching or group loading.*

After creation, a class or interface is determined not by its name alone, but by a pair: its binary name (§4.2.1) and its defining loader. Each such class or interface belongs to a single *run-time package*. The run-time package of a class or interface is determined by the package name and the defining loader of the class or interface.

5.3.1. Loading Using the Bootstrap Class Loader

The process of loading and creating the nonarray class or interface C denoted by N using the bootstrap class loader is as follows.

First, the Java Virtual Machine determines whether the bootstrap class loader has already been recorded as an initiating loader of a class or interface denoted by N. If so, this class or interface is C, and no class loading or creation is necessary.

Otherwise, the Java Virtual Machine passes the argument N to an invocation of a method on the bootstrap class loader. To load C, the bootstrap class loader locates a purported representation of C in a platform-dependent manner, then asks the Java Virtual Machine to derive a class or interface C denoted by N from the purported representation using the bootstrap class loader, and then to create C, via the algorithm of §5.3.5.

Typically, a class or interface will be represented using a file in a hierarchical file system, and the name of the class or interface will be encoded in the pathname of the file to aid in locating it.

If no purported representation of C is found, the bootstrap class loader throws a `ClassNotFoundException`. The process of loading and creating C then fails with a `NoClassDefFoundError` whose cause is the `ClassNotFoundException`.

If a purported representation of C is found, but deriving C from the purported representation fails, then the process of loading and creating C fails for the same reason.

Otherwise, the process of loading and creating C succeeds.

5.3.2. Loading Using a User-defined Class Loader

The process of loading and creating the nonarray class or interface C denoted by N using a user-defined class loader L is as follows.

First, the Java Virtual Machine determines whether L has already been recorded as an initiating loader of a class or interface denoted by N. If so, this class or interface is C, and no class loading or creation is necessary.

Otherwise, the Java Virtual Machine invokes the `loadClass` method of class `ClassLoader` on L, passing the name N of a class or interface. L must perform one of the following two operations to load and thereby create a class or interface C:

1. The class loader L can load C directly. This is accomplished by obtaining an array of bytes that purports to represent C as a `ClassFile` structure (§4.1), and then invoking the method `defineClass` of class `ClassLoader`. Invoking `defineClass` causes the Java Virtual Machine to derive a class or interface C denoted by N from the array of bytes using L, and then to create C, via the algorithm of §5.3.5. L should use the result of `defineClass` as the result of `loadClass`.
2. The class loader L can load C indirectly, by delegating the loading of C to some other class loader L'. This is accomplished by passing the argument N to an invocation of a method on L' (typically the `loadClass` method of class `ClassLoader`). L should use the result of that method as the result of `loadClass`.

The following rules apply regardless of which operation is performed:

- **If a class loader cannot find a purported representation of a class or interface denoted by N, it must throw a `ClassNotFoundException`. The process of loading and creating C then fails with a `NoClassDefFoundError` whose cause is the `ClassNotFoundException`.**

- **If a class loader finds a purported representation of C, but deriving C from the purported representation fails, then the process of loading and creating C fails for the same reason.**
- **If a class loader throws an exception other than a `ClassNotFoundException`, then the process of loading and creating C fails for the same reason.**

If the invocation of `loadClass` on `L` has a result, then:

- **If the result is `null`, or the result is a class or interface with a name other than `N`, then the result is discarded, and the process of loading and creation fails with a `NoClassDefFoundError`.**
- Otherwise, the result is the created class or interface C. The Java Virtual Machine records that `L` is an initiating loader of C (§5.3.4). The process of loading and creating C succeeds.

Since JDK 1.1, Oracle's Java Virtual Machine implementation has invoked the one-argument `loadClass` method on a class loader to cause it to load a class or interface. The argument to `loadClass` is the name of the class or interface to be loaded. There is also a two-argument version of the `loadClass` method, where the second argument is a `boolean` that indicates whether the class or interface is to be linked or not. Only the two-argument version was supplied in JDK 1.0.2, and Oracle's Java Virtual Machine implementation relied on it to link the loaded class or interface. From JDK 1.1 onward, Oracle's Java Virtual Machine implementation links the class or interface directly, without relying on the class loader.

5.3.3. Creating Array Classes

The following steps are used to create the array class C denoted by the name `N` in association with the class loader `L`. `L` may be either the bootstrap class loader or a user-defined class loader.

First, the Java Virtual Machine determines whether `L` has already been recorded as an initiating loader of an array class with the same component type as `N`. If so, this class is C, and no array class creation is necessary.

Otherwise, the following steps are performed to create C:

1. If the component type is a `reference` type, the algorithm of this section (§5.3) is applied recursively using `L` in order to load and thereby create the component type of C.
2. The Java Virtual Machine creates a new array class with the indicated component type and number of dimensions.

   If the component type is a `reference` type, the Java Virtual Machine marks C to have the defining loader of the component type as its defining loader. Otherwise, the Java Virtual Machine marks C to have the bootstrap class loader as its defining loader.

   In any case, the Java Virtual Machine then records that `L` is an initiating loader for C (§5.3.4).

   If the component type is a `reference` type, the accessibility of the array class is determined by the accessibility of its component type (§5.4.4). Otherwise, the array class is accessible to all classes and interfaces.

5.3.4. Loading Constraints

Ensuring type safe linkage in the presence of class loaders requires special care. It is possible that when two different class loaders initiate loading of a class or interface denoted by `N`, the name `N` may denote a different class or interface in each loader.

When a class or interface C = $\langle N_1, L_1 \rangle$ makes a symbolic reference to a field or method of another class or interface D = $\langle N_2, L_2 \rangle$, the symbolic reference includes a descriptor specifying the type of the field, or the return and argument types of the method. It is essential that any class or interface name `N` mentioned in the field or method descriptor (§4.3.2, §4.3.3) denote the same class or interface when loaded by L_1 and when loaded by L_2.

To ensure this, the Java Virtual Machine imposes *loading constraints* of the form $N^{L_1} = N^{L_2}$ during preparation (§5.4.2) and resolution (§5.4.3). To enforce these constraints, the Java Virtual Machine will, at certain prescribed times (see §5.3.1, §5.3.2, §5.3.3, and §5.3.5), record that a particular loader is an initiating loader of a particular class. After recording that a loader is an initiating loader of a class, the Java Virtual Machine must immediately check to see if any loading constraints are violated. If so, the record is retracted, the Java Virtual Machine throws a `LinkageError`, and the loading operation that caused the recording to take place fails.

Similarly, after imposing a loading constraint (see §5.4.2, §5.4.3.2, §5.4.3.3, and §5.4.3.4), the Java Virtual Machine must immediately check to see if any loading constraints are violated. If so, the newly imposed loading constraint is retracted, the Java Virtual Machine throws a `LinkageError`, and the operation that caused the constraint to be imposed (either resolution or preparation, as the case may be) fails.

The situations described here are the only times at which the Java Virtual Machine checks whether any loading constraints have been violated. A loading constraint is violated if, and only if, all the following four conditions hold:

- There exists a loader `L` such that `L` has been recorded by the Java Virtual Machine as an initiating loader of a class C named `N`.
- There exists a loader `L'` such that `L'` has been recorded by the Java Virtual Machine as an initiating loader of a class C ' named `N`.
- The equivalence relation defined by the (reflexive, transitive closure of the) set of imposed constraints implies $N^L = N^{L'}$.
- C ≠ C '.

A full discussion of class loaders and type safety is beyond the scope of this specification. For a more comprehensive discussion, readers are referred to Dynamic Class Loading in the Java Virtual Machine by Sheng Liang and Gilad Bracha (Proceedings of the 1998 ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications).

5.3.5. Deriving a Class from a `class` File Representation

Class loaders require the cooperation of the Java Virtual Machine to derive and create a class or interface from a binary representation provided by the loader (§5.3.1, §5.3.2). The following steps are used to derive a class or interface C denoted by `N` from a purported representation in `class` file format when requested by a class loader `L`.

1. First, the Java Virtual Machine determines whether the request by class loader `L` to derive a class or interface denoted by `N` is permitted.

   If `L` has already been recorded as an initiating loader of a class or interface denoted by `N`, derivation throws a `LinkageError`.

If the Java Virtual Machine is already in the process of deriving a class or interface denoted by `N` as requested by class loader `L`, derivation throws a `ClassCircularityError`.

2. Next, the Java Virtual Machine attempts to parse the purported representation. The purported representation may not in fact be a valid representation of C, so derivation must detect the following problems:

   - **If the purported representation is not a `ClassFile` structure (§4.1, §4.8), derivation throws a `ClassFormatError`.**

   - **Otherwise, if the purported representation is not of a supported major or minor version (§4.1), derivation throws an `UnsupportedClassVersionError`.**

     `UnsupportedClassVersionError`, a subclass of `ClassFormatError`, was introduced in JDK 1.2 to enable easy identification of a `ClassFormatError` caused by an attempt to load a class whose representation uses an unsupported version of the `class` file format. In JDK 1.1 and earlier, an instance of `NoClassDefFoundError` or `ClassFormatError` was thrown in case of an unsupported version, depending on whether the class was being loaded by the system class loader or a user-defined class loader.

   - **Otherwise, if the purported representation does not actually represent a class or interface named `N`, derivation throws a `NoClassDefFoundError`.**

     This occurs when the purported representation has either a `this_class` item which specifies a name other than `N`, or an `access_flags` item which has the `ACC_MODULE` flag set.

3. If C has a direct superclass, the symbolic reference from C to its direct superclass is resolved using the algorithm of §5.4.3.1, with `L` acting as the defining loader of C. Note that if C is an interface it must have `Object` as its direct superclass, which must already have been loaded. Only `Object` has no direct superclass.

   Any exception that can be thrown as a result of failure of class or interface resolution can be thrown as a result of derivation. In addition, derivation must detect the following problems:

   - **If the class or interface named as the direct superclass of C is in fact an interface or a `final` class, derivation throws an `IncompatibleClassChangeError`.**

   - **Otherwise, if the class named as the direct superclass of C has a `PermittedSubclasses` attribute (§4.7.31) and any of the following is true, derivation throws an `IncompatibleClassChangeError`:**

     - **The superclass is in a different run-time module than C (§5.3.6).**

     - **C does not have its `ACC_PUBLIC` flag set (§4.1) and the superclass is in a different run-time package than C (§5.3).**

     - **No entry in the `classes` array of the superclass's `PermittedSubclasses` attribute refers to a class or interface with the name `N`.**

   - **Otherwise, if C is a class and some instance method declared in C can override (§5.4.5) a `final` instance method declared in a superclass of C, derivation throws an `IncompatibleClassChangeError`.**

4. If C has any direct superinterfaces, the symbolic references from C to its direct superinterfaces are resolved using the algorithm of §5.4.3.1, with `L` acting as the defining loader of C.

   Any exception that can be thrown as a result of failure of class or interface resolution can be thrown as a result of derivation. In addition, derivation must detect the following problems:

   - **If any class or interface named as a direct superinterface of C is not in fact an interface, derivation throws an `IncompatibleClassChangeError`.**

   - **Otherwise, for each direct superinterface named by C, if the superinterface has a `PermittedSubclasses` attribute (§4.7.31) and any of the following is true, derivation throws an `IncompatibleClassChangeError`:**

     - **The superinterface is in a different run-time module than C.**

     - **C does not have its `ACC_PUBLIC` flag set (§4.1) and the superinterface is in a different run-time package than C.**

     - **No entry in the `classes` array of the superinterface's `PermittedSubclasses` attribute refers to a class or interface with the name `N`.**

If no exception is thrown in steps 1-4, then derivation of the class or interface C succeeds. The Java Virtual Machine marks C to have `L` as its defining loader, records that `L` is an initiating loader of C (§5.3.4), and creates C in the method area (§2.5.4).

When derivation succeeds, the process of loading and creating C is not complete until every class loader that was involved in loading C (directly or indirectly) returns C as its result. Depending on the behavior of user-defined class loaders, the process of loading and creating C may yet fail (§5.3.2).

If an exception is thrown in steps 1-4, then derivation of the class or interface C fails with that exception.

Requests to derive a class or interface may be made concurrently by class loader code executing in multiple threads, but the derivation process is sequential. The Java Virtual Machine implementation ensures that only one request by a given class loader to derive a class or interface of a given name is processed at a time, while all other such requests wait until the first request is complete.

As specified by the derivation process, if the first request is successful, no subsequent requests will be permitted. The `ClassLoader` API provides mechanisms to synchronize derivation requests and cache successful results so that redundant derivation requests do not occur.

5.3.6. Modules and Layers

The Java Virtual Machine supports the organization of classes and interfaces into modules. The membership of a class or interface C in a module `M` is used to control access to C from classes and interfaces in modules other than `M` (§5.4.4).

Module membership is defined in terms of run-time packages (§5.3). A program determines the names of the packages in each module, and the class loaders that will create the classes and interfaces of the named packages; it then specifies the packages and class loaders to an invocation of the `defineModules` method of the class `ModuleLayer`. Invoking `defineModules` causes the Java Virtual Machine to create new *run-time modules* that are associated with the run-time packages of the class loaders.

Every run-time module indicates the run-time packages that it *exports*, which influences access to the `public` classes and interfaces in those run-time packages. Every run-time module also indicates the other run-time modules that it *reads*, which influences access by its own code to the `public` types and interfaces in those run-time modules.

We say that *a class is in a run-time module* iff the class's run-time package is associated (or will be associated, if the class is actually created) with that run-time module.

A class created by a class loader is in exactly one run-time package and therefore exactly one run-time module, because the Java Virtual Machine does not support a run-time package being associated with (or more evocatively, "split across") multiple run-time modules.

A run-time module is implicitly bound to exactly one class loader, by the semantics of `defineModules`. On the other hand, a class loader may create classes in more than one run-time module, because the Java Virtual Machine does not require all the run-time packages of a class loader to be associated with the same run-time module.

In other words, the relationship between class loaders and run-time modules need not be 1:1. For a given set of modules to be loaded, if a program can determine that the names of the packages in each module are found only in that module, then the program may specify only one class loader to the invocation of `defineModules`. This class loader will create classes across multiple run-time modules.

Every run-time module created by `defineModules` is part of a *layer*. A layer represents a set of class loaders that jointly serve to create classes in a set of run-time modules. There are two kinds of layers: the boot layer supplied by the Java Virtual Machine, and user-defined layers. The boot layer is created at Java Virtual Machine startup in an implementation-dependent manner. It associates the standard run-time module `java.base` with standard run-time packages defined by the bootstrap class loader, such as `java.lang`. User-defined layers are created by programs in order to construct sets of run-time modules that depend on `java.base` and other standard run-time modules.

A run-time module is implicitly part of exactly one layer, by the semantics of `defineModules`. However, a class loader may create classes in the run-time modules of different layers, because the same class loader may be specified to multiple invocations of `defineModules`. Access control is governed by a class's run-time module, not by the class loader which created the class or by the layer(s) which the class loader serves.

The set of class loaders specified for a layer, and the set of run-time modules which are part of a layer, are immutable after the layer is created. However, the `ModuleLayer` class affords programs a degree of dynamic control over the relationships between the run-time modules in a user-defined layer.

If a user-defined layer contains more than one class loader, then any delegation between the class loaders is the responsibility of the program that created the layer. The Java Virtual Machine does not check that the layer's class loaders delegate to each other in accordance with how the layer's run-time modules read each other. Moreover, if the layer's run-time modules are modified via the `ModuleLayer` class to read additional run-time modules, then the Java Virtual Machine does not check that the layer's class loaders are modified by some out-of-band mechanism to delegate in a corresponding fashion.

There are similarities and differences between class loaders and layers. On the one hand, a layer is similar to a class loader in that each may delegate to, respectively, one or more parent layers or class loaders that created, respectively, modules or classes at an earlier time. That is, the set of modules specified to a layer may depend on modules not specified to the layer, and instead specified previously to one or more parent layers. On the other hand, a layer may be used to create new modules only once, whereas a class loader may be used to create new classes or interfaces at any time via multiple invocations of the `defineClass` method.

It is possible for a class loader to define a class or interface in a run-time package that was not associated with a run-time module by any of the layers which the class loader serves. This may occur if the run-time package embodies a named package that was not specified to `defineModules`, or if the class or interface has a simple binary name (§4.2.1) and thus is a member of a run-time package that embodies an unnamed package (JLS §7.4.2). In either case, the class or interface is treated as a member of a special run-time module which is implicitly bound to the class loader. This special run-time module is known as the *unnamed module* of the class loader. The run-time package of the class or interface is associated with the unnamed module of the class loader. There are special rules for unnamed modules, designed to maximize their interoperation with other run-time modules, as follows:

- A class loader's unnamed module is distinct from all other run-time modules bound to the same class loader.
- A class loader's unnamed module is distinct from all run-time modules (including unnamed modules) bound to other class loaders.
- Every unnamed module reads every run-time module.
- Every unnamed module exports, to every run-time module, every run-time package associated with itself.

5.4. Linking

Linking a class or interface involves verifying and preparing that class or interface, its direct superclass, its direct superinterfaces, and its element type (if it is an array type), if necessary. Linking also involves resolution of symbolic references in the class or interface, though not necessarily at the same time as the class or interface is verified and prepared.

This specification allows an implementation flexibility as to when linking activities (and, because of recursion, loading) take place, provided that all of the following properties are maintained:

- A class or interface is completely loaded before it is linked.
- A class or interface is completely verified and prepared before it is initialized.
- Errors detected during linkage are thrown at a point in the program where some action is taken by the program that might, directly or indirectly, require linkage to the class or interface involved in the error.
- A symbolic reference to a dynamically-computed constant is not resolved until either (i) an *ldc*, *ldc_w*, or *ldc2_w* instruction that refers to it is executed, or (ii) a bootstrap method that refers to it as a static argument is invoked.

  A symbolic reference to a dynamically-computed call site is not resolved until a bootstrap method that refers to it as a static argument is invoked.

For example, a Java Virtual Machine implementation may choose a "lazy" linkage strategy, where each symbolic reference in a class or interface (other than the symbolic references above) is resolved individually when it is used. Alternatively, an implementation may choose an "eager" linkage strategy, where all symbolic references are resolved at once when the class or interface is being verified. This means that the resolution process may continue, in some implementations, after a class or interface has been initialized. Whichever strategy is followed, any error detected during resolution must be thrown at a point in the program that (directly or indirectly) uses a symbolic reference to the class or interface.

Because linking involves the allocation of new data structures, it may fail with an `OutOfMemoryError`.

5.4.1. Verification

Verification (§4.10) ensures that the binary representation of a class or interface is structurally correct (§4.9). Verification may cause additional classes and interfaces to be loaded (§5.3) but need not cause them to be verified or prepared.

If the binary representation of a class or interface does not satisfy the static or structural constraints listed in §4.9, then a `VerifyError` must be thrown at the point in the program that caused the class or interface to be verified.

If an attempt by the Java Virtual Machine to verify a class or interface fails because an error is thrown that is an instance of `LinkageError` (or a subclass), then subsequent attempts to verify the class or interface always fail with the same error that was thrown as a result of the initial verification attempt.

5.4.2. Preparation

Preparation involves creating the static fields for a class or interface and initializing such fields to their default values (§2.3, §2.4). This does not require the execution of any Java Virtual Machine code; explicit initializers for static fields are executed as part of initialization (§5.5), not preparation.

During preparation of a class or interface C, the Java Virtual Machine also imposes loading constraints (§5.3.4):

1. Let L_1 be the defining loader of C. For each instance method `m` declared in C that can override (§5.4.5) an instance method declared in a superclass or superinterface D = $<N_2, L_2>$, for each class or interface name `N` mentioned by the descriptor of `m` (§4.3.3), the Java Virtual Machine imposes the loading constraint $N^{L_1} = N^{L_2}$.

2. For each instance method `m` declared in a superinterface I = $<N_3, L_3>$ of C, if C does not itself declare an instance method that can override `m`, then a method is selected (§5.4.6) with respect to C and the method `m` in I. Let D = $<N_2, L_2>$ be the class or interface that declares the selected method. For each class or interface name `N` mentioned by the descriptor of `m`, the Java Virtual Machine imposes the loading constraint $N^{L_2} = N^{L_3}$.

Preparation may occur at any time following creation but must be completed prior to initialization.

5.4.3. Resolution

Many Java Virtual Machine instructions - *anewarray*, *checkcast*, *getfield*, *getstatic*, *instanceof*, *invokedynamic*, *invokeinterface*, *invokespecial*, *invokestatic*, *invokevirtual*, *ldc*, *ldc_w*, *ldc2_w*, *multianewarray*, *new*, *putfield*, and *putstatic* - rely on symbolic references in the run-time constant pool. Execution of any of these instructions requires *resolution* of the symbolic reference.

Resolution is the process of dynamically determining one or more concrete values from a symbolic reference in the run-time constant pool. Initially, all symbolic references in the run-time constant pool are unresolved.

Resolution of an unresolved symbolic reference to (i) a class or interface, (ii) a field, (iii) a method, (iv) a method type, (v) a method handle, or (vi) a dynamically-computed constant, proceeds in accordance with the rules given in §5.4.3.1 through §5.4.3.5. In the first three of those sections, the class or interface in whose run-time constant pool the symbolic reference appears is labeled D. Then:

- **If no error occurs during resolution of the symbolic reference, then resolution succeeds.**

  Subsequent attempts to resolve the symbolic reference always succeed trivially and result in the same entity produced by the initial resolution. If the symbolic reference is to a dynamically-computed constant, the bootstrap method is not re-executed for these subsequent attempts.

- **If an error occurs during resolution of the symbolic reference, then it is either (i) an instance of `IncompatibleClassChangeError` (or a subclass); (ii) an instance of `Error` (or a subclass) that arose from resolution or invocation of a bootstrap method; or (iii) an instance of `LinkageError` (or a subclass) that arose because class loading failed or a loader constraint was violated. The error must be thrown at a point in the program that (directly or indirectly) uses the symbolic reference.**

  Subsequent attempts to resolve the symbolic reference always fail with the same error that was thrown as a result of the initial resolution attempt. If the symbolic reference is to a dynamically-computed constant, the bootstrap method is not re-executed for these subsequent attempts.

Because errors occurring on an initial attempt at resolution are thrown again on subsequent attempts, a class in one module that attempts to access, via resolution of a symbolic reference in its run-time constant pool, an unexported `public` type in a different module will always receive the same error indicating an inaccessible type (§5.4.4), even if the Java SE Platform API is used to dynamically export the `public` type's package at some time after the class's first attempt.

Resolution of an unresolved symbolic reference to a dynamically-computed call site proceeds in accordance with the rules given in §5.4.3.6. Then:

- If no error occurs during resolution of the symbolic reference, then resolution succeeds *solely for the instruction in the `class` file that required resolution*. This instruction necessarily has an opcode of *invokedynamic*.

  Subsequent attempts to resolve the symbolic reference *by that instruction in the `class` file* always succeed trivially and result in the same entity produced by the initial resolution. The bootstrap method is not re-executed for these subsequent attempts.

  The symbolic reference is still unresolved for all other instructions in the `class` file, of any opcode, which indicate the same entry in the run-time constant pool as the *invokedynamic* instruction above.

- If an error occurs during resolution of the symbolic reference, then it is either (i) an instance of `IncompatibleClassChangeError` (or a subclass); (ii) an instance of `Error` (or a subclass) that arose from resolution or invocation of a bootstrap method; or (iii) an instance of `LinkageError` (or a subclass) that arose because class loading failed or a loader constraint was violated. The error must be thrown at a point in the program that (directly or indirectly) uses the symbolic reference.

  Subsequent attempts *by the same instruction in the `class` file* to resolve the symbolic reference always fail with the same error that was thrown as a result of the initial resolution attempt. The bootstrap method is not re-executed for these subsequent attempts.

  The symbolic reference is still unresolved for all other instructions in the `class` file, of any opcode, which indicate the same entry in the run-time constant pool as the *invokedynamic* instruction above.

Certain of the instructions above require additional linking checks when resolving symbolic references. For instance, in order for a *getfield* instruction to successfully resolve the symbolic reference to the field on which it operates, it must not only complete the field resolution steps given in §5.4.3.2 but also check that the field is not `static`. If it is a `static` field, a linking exception must be thrown.

Linking exceptions generated by checks that are specific to the execution of a particular Java Virtual Machine instruction are given in the description of that instruction and are not covered in this general discussion of resolution. Note that such exceptions, although described as part of the execution of Java Virtual Machine instructions rather than resolution, are still properly considered failures of resolution.

5.4.3.1. Class and Interface Resolution

To resolve an unresolved symbolic reference from D to a class or interface C denoted by `N`, the following steps are performed:

1. The defining loader of D is used to load and thereby create a class or interface denoted by `N`. This class or interface is C. The details of the process are given in §5.3.

   Any exception that can be thrown as a result of failure to load and thereby create C can thus be thrown as a result of failure of class and interface resolution.

2. If C is an array class and its element type is a `reference` type, then a symbolic reference to the class or interface representing the element type is resolved by invoking the algorithm in §5.4.3.1 recursively.

3. Finally, access control is applied for the access from D to C (§5.4.4).

If steps 1 and 2 succeed but step 3 fails, C is still valid and usable. Nevertheless, resolution fails, and D is prohibited from accessing C.

5.4.3.2. Field Resolution

To resolve an unresolved symbolic reference from D to a field in a class or interface C, the symbolic reference to C given by the field reference must first be resolved (§5.4.3.1). Therefore, any exception that can be thrown as a result of failure of resolution of a class or interface reference can be thrown as a result of failure of field resolution. If the reference to C can be successfully resolved, an exception relating to the failure of resolution of the field reference itself can be thrown.

When resolving a field reference, field resolution first attempts to look up the referenced field in C and its superclasses:

1. If C declares a field with the name and descriptor specified by the field reference, field lookup succeeds. The declared field is the result of the field lookup.

2. Otherwise, field lookup is applied recursively to the direct superinterfaces of the specified class or interface C.

3. Otherwise, if C has a superclass S, field lookup is applied recursively to S.

4. Otherwise, field lookup fails.

Then, the result of field resolution is determined:

- **If field lookup failed, field resolution throws a `NoSuchFieldError`.**
- Otherwise, field lookup succeeded. Access control is applied for the access from D to the field which is the result of field lookup (§5.4.4). Then:
  - **If access control failed, field resolution fails for the same reason.**
  - Otherwise, access control succeeded. Loading constraints are imposed, as follows.

    Let <E, L_1> be the class or interface in which the referenced field is actually declared. Let L_2 be the defining loader of D.

    For any class or interface name `N` mentioned by the descriptor of the referenced field (§4.3.2), the Java Virtual Machine imposes the loading constraint $N^{L_1} = N^{L_2}$ (§5.3.4).

    If imposing this constraint results in any loading constraints being violated, then field resolution fails. Otherwise, field resolution succeeds.

5.4.3.3. Method Resolution

To resolve an unresolved symbolic reference from D to a method in a class C, the symbolic reference to C given by the method reference is first resolved (§5.4.3.1). Therefore, any exception that can be thrown as a result of failure of resolution of a class reference can be thrown as a result of failure of method resolution. If the reference to C can be successfully resolved, exceptions relating to the resolution of the method reference itself can be thrown.

When resolving a method reference:

1. **If C is an interface, method resolution throws an `IncompatibleClassChangeError`.**

2. Otherwise, method resolution attempts to locate the referenced method in C and its superclasses:

   - If C declares exactly one method with the name specified by the method reference, and the declaration is a signature polymorphic method (§2.9.3), then method lookup succeeds. The descriptor specified by the method reference is resolved, as if by resolution of an unresolved symbolic reference to a method type (§5.4.3.5).

     The resolved method is the signature polymorphic method declaration. It is not necessary for C to declare a method with the descriptor specified by the method reference.

   - Otherwise, if C declares a method with the name and descriptor specified by the method reference, method lookup succeeds.

   - Otherwise, if C has a superclass, step 2 of method resolution is recursively invoked on the direct superclass of C.

3. Otherwise, method resolution attempts to locate the referenced method in the superinterfaces of the specified class C:

   - If the *maximally-specific superinterface methods* of C for the name and descriptor specified by the method reference include exactly one method that does not have its `ACC_ABSTRACT` flag set, then this method is chosen and method lookup succeeds.

   - Otherwise, if any superinterface of C declares a method with the name and descriptor specified by the method reference that has neither its `ACC_PRIVATE` flag nor its `ACC_STATIC` flag set, one of these is arbitrarily chosen and method lookup succeeds.

   - Otherwise, method lookup fails.

A *maximally-specific superinterface method* of a class or interface C for a particular method name and descriptor is any method for which all of the following are true:

- The method is declared in a superinterface (direct or indirect) of C.
- The method is declared with the specified name and descriptor.
- The method has neither its `ACC_PRIVATE` flag nor its `ACC_STATIC` flag set.
- Where the method is declared in interface I, there exists no other maximally-specific superinterface method of C with the specified name and descriptor that is declared in a subinterface of I.

The result of method resolution is determined as follows:

- **If method lookup failed, method resolution throws a `NoSuchMethodError`.**
- Otherwise, method lookup succeeded. Access control is applied for the access from D to the method which is the result of method lookup (§5.4.4). Then:
  - **If access control failed, method resolution fails for the same reason.**
  - Otherwise, access control succeeded. Loading constraints are imposed, as follows.

    Let <E, L_1> be the class or interface in which the referenced method `m` is actually declared. Let L_2 be the defining loader of D.

    For each class or interface name `N` mentioned by the descriptor of the referenced method (§4.3.3), the Java Virtual Machine imposes the loading constraint $N^{L_1} = N^{L_2}$ (§5.3.4).

    If imposing these constraints results in any loading constraints being violated, then method resolution fails. Otherwise, method resolution succeeds.

When resolution searches for a method in the class's superinterfaces, the best outcome is to identify a maximally-specific non-`abstract` method. It is possible that this method will be chosen by method selection, so it is desirable to add class loader constraints for it.

Otherwise, the result is nondeterministic. This is not new: The Java® Virtual Machine Specification has never identified exactly which method is chosen, and how "ties" should be broken. Prior to Java SE 8, this was mostly an unobservable distinction. However, beginning with Java SE 8, the set of interface methods is more heterogenous, so care must be taken to avoid problems with nondeterministic behavior. Thus:

- *Superinterface methods that are `private` and `static` are ignored by resolution. This is consistent with the Java programming language, where such interface methods are not inherited.*
- *Any behavior controlled by the resolved method should not depend on whether the method is `abstract` or not.*

Note that if the result of resolution is an `abstract` method, the referenced class C may be non-`abstract`. Requiring C to be `abstract` would conflict with the nondeterministic choice of superinterface methods. Instead, resolution assumes that the run time class of the invoked object has a concrete implementation of the method.

5.4.3.4. Interface Method Resolution

To resolve an unresolved symbolic reference from D to an interface method in an interface C, the symbolic reference to C given by the interface method reference is first resolved (§5.4.3.1). Therefore, any exception that can be thrown as a result of failure of resolution of an interface reference can be thrown as a result of failure of interface method resolution. If the reference to C can be successfully resolved, exceptions relating to the resolution of the interface method reference itself can be thrown.

When resolving an interface method reference:

1. **If C is not an interface, interface method resolution throws an `IncompatibleClassChangeError`.**
2. Otherwise, if C declares a method with the name and descriptor specified by the interface method reference, method lookup succeeds.
3. Otherwise, if the class `Object` declares a method with the name and descriptor specified by the interface method reference, which has its `ACC_PUBLIC` flag set and does not have its `ACC_STATIC` flag set, method lookup succeeds.
4. Otherwise, if the maximally-specific superinterface methods (§5.4.3.3) of C for the name and descriptor specified by the method reference include exactly one method that does not have its `ACC_ABSTRACT` flag set, then this method is chosen and method lookup succeeds.
5. Otherwise, if any superinterface of C declares a method with the name and descriptor specified by the method reference that has neither its `ACC_PRIVATE` flag nor its `ACC_STATIC` flag set, one of these is arbitrarily chosen and method lookup succeeds.
6. Otherwise, method lookup fails.

The result of interface method resolution is determined as follows:

- **If method lookup failed, interface method resolution throws a `NoSuchMethodError`.**
- **Otherwise, method lookup succeeded. Access control is applied for the access from D to the method which is the result of method lookup (§5.4.4). Then:**
  - **If access control failed, interface method resolution fails for the same reason.**
  - Otherwise, access control succeeded. Loading constraints are imposed, as follows.

    Let <E, L_1> be the class or interface in which the referenced interface method `m` is actually declared. Let L_2 be the defining loader of D.

    For each class or interface name `N` mentioned by the descriptor of the referenced method (§4.3.3), the Java Virtual Machine imposes the loading constraint $N^{L_1} = N^{L_2}$ (§5.3.4).

    If imposing these constraints results in any loading constraints being violated, then interface method resolution fails. Otherwise, interface method resolution succeeds.

Access control is necessary because interface method resolution may pick a `private` method of interface C. (Prior to Java SE 8, the result of interface method resolution could be a non-`public` method of class `Object` or a `static` method of class `Object`; such results were not consistent with the inheritance model of the Java programming language, and are disallowed in Java SE 8 and above.)

5.4.3.5. Method Type and Method Handle Resolution

To resolve an unresolved symbolic reference to a method type, it is as if resolution occurs of unresolved symbolic references to the classes and interfaces (§5.4.3.1) whose names are mentioned by the method descriptor (§4.3.3), in the order in which they are mentioned.

Any exception that can be thrown as a result of failure of resolution of a reference to a class or interface can thus be thrown as a result of failure of method type resolution.

The result of successful method type resolution is a `reference` to an instance of `java.lang.invoke.MethodType` which represents the method descriptor.

Method type resolution occurs regardless of whether the run-time constant pool actually contains symbolic references to classes and interfaces indicated in the method descriptor. Also, the resolution is deemed to occur on unresolved symbolic references, so a failure to resolve one method type will not necessarily lead to a later failure to resolve another method type with the same textual method descriptor, if suitable classes and interfaces can be loaded by the later time.

Resolution of an unresolved symbolic reference to a method handle is more complicated. Each method handle resolved by the Java Virtual Machine has an equivalent instruction sequence called its *bytecode behavior*, indicated by the method handle's *kind*. The integer values and descriptions of the nine kinds of method handle are given in Table 5.4.3.5-A.

Symbolic references by an instruction sequence to fields or methods are indicated by `C.x:T`, where `x` and `T` are the name and descriptor (§4.3.2, §4.3.3) of the field or method, and `C` is the class or interface in which the field or method is to be found.

Table 5.4.3.5-A. Bytecode Behaviors for Method Handles

Kind	Description	Interpretation
1	`REF_getField`	`getfield C.f:T`
2	`REF_getStatic`	`getstatic C.f:T`
3	`REF_putField`	`putfield C.f:T`
4	`REF_putStatic`	`putstatic C.f:T`
5	`REF_invokeVirtual`	`invokevirtual C.m:(A*)T`
6	`REF_invokeStatic`	`invokestatic C.m:(A*)T`
7	`REF_invokeSpecial`	`invokespecial C.m:(A*)T`
8	`REF_newInvokeSpecial`	`new C; dup; invokespecial C.<init>:(A*)V`
9	`REF_invokeInterface`	`invokeinterface C.m:(A*)T`

Let `MH` be the symbolic reference to a method handle (§5.1) being resolved. Also:

- Let R be the symbolic reference to a field or method given by `MH`.

  For example, R is a symbolic reference to `C.f` for bytecode behavior of kind 1, and a symbolic reference to `C.<init>` for bytecode behavior of kind 8.

- Let C be the class, interface, or array type referenced by R.
- Let T be the type given by the field descriptor of R, or the return type given by the method descriptor of R. Let A* be the sequence (perhaps empty) of parameter types given by the method descriptor of R.

To resolve `MH`, all symbolic references to classes, interfaces, fields, and methods in `MH`'s bytecode behavior are resolved, using the following three steps:

1. R is resolved. This occurs as if by field resolution (§5.4.3.2) when `MH`'s bytecode behavior is kind 1, 2, 3, or 4, and as if by method resolution (§5.4.3.3) when `MH`'s bytecode behavior is kind 5, 6, 7, or 8, and as if by interface method resolution (§5.4.3.4) when `MH`'s bytecode behavior is kind 9.
2. The following constraints apply to the result of resolving R. These constraints correspond to those that would be enforced during verification or execution of the instruction sequence for the relevant bytecode behavior.
   - If `MH`'s bytecode behavior is kind 7 (`REF_invokeSpecial`), then C must be the current class or interface, a superclass of the current class, a direct superinterface of the current class or interface, or `Object`.
   - If `MH`'s bytecode behavior is kind 8 (`REF_newInvokeSpecial`), then R must resolve to an instance initialization method declared in class C.
   - If R resolves to a `protected` member, then the following rules apply depending on the kind of `MH`'s bytecode behavior:
     - For kinds 1, 3, and 5 (`REF_getField`, `REF_putField`, and `REF_invokeVirtual`): If `C.f` or `C.m` resolved to a `protected` field or method, and C is in a different run-time package than the current class, then C must be a subclass of the current class.
     - For kind 8 (`REF_newInvokeSpecial`): If `C.<init>` resolved to a `protected` method, then C must be declared in the same run-time package as the current class.
   - R must resolve to a `static` or non-`static` member depending on the kind of `MH`'s bytecode behavior:
     - For kinds 1, 3, 5, 7, and 9 (`REF_getField`, `REF_putField`, `REF_invokeVirtual`, `REF_invokeSpecial`, and `REF_invokeInterface`): `C.f` or `C.m` must resolve to a non-`static` field or method.
     - For kinds 2, 4, and 6 (`REF_getStatic`, `REF_putStatic`, and `REF_invokeStatic`): `C.f` or `C.m` must resolve to a `static` field or method.
3. A reference to an instance of `java.lang.invoke.MethodType` is obtained as if by resolution of an unresolved symbolic reference to a method type that contains the method descriptor specified in Table 5.4.3.5-B for the kind of `MH`.

   It is as if the symbolic reference to a method handle contains a symbolic reference to the method type that the resolved method handle will eventually have. The detailed structure of the method type is obtained by inspecting Table 5.4.3.5-B.

   Table 5.4.3.5-B. Method Descriptors for Method Handles

Kind	Description	Method descriptor
1	`REF_getField`	`(C)T`
2	`REF_getStatic`	`()T`

Kind	Description	Method descriptor
3	REF_putField	(C,T)V
4	REF_putStatic	(T)V
5	REF_invokeVirtual	(C,A*)T
6	REF_invokeStatic	(A*)T
7	REF_invokeSpecial	(C,A*)T
8	REF_newInvokeSpecial	(A*)C
9	REF_invokeInterface	(C,A*)T

In steps 1 and 3, any exception that can be thrown as a result of failure of resolution of a symbolic reference to a class, interface, field, or method can be thrown as a result of failure of method handle resolution. In step 2, any failure due to the specified constraints causes a failure of method handle resolution due to an `IllegalAccessError`.

The intent is that resolving a method handle can be done in exactly the same circumstances that the Java Virtual Machine would successfully verify and resolve the symbolic references in the bytecode behavior. In particular, method handles to `private`, `protected`, and `static` members can be created in exactly those classes for which the corresponding normal accesses are legal.

The result of successful method handle resolution is a `reference` to an instance of `java.lang.invoke.MethodHandle` which represents the method handle `MH`.

The type descriptor of this `java.lang.invoke.MethodHandle` instance is the `java.lang.invoke.MethodType` instance produced in the third step of method handle resolution above.

The type descriptor of a method handle is such that a valid call to `invokeExact` in `java.lang.invoke.MethodHandle` on the method handle has exactly the same stack effects as the bytecode behavior. Calling this method handle on a valid set of arguments has exactly the same effect and returns the same result (if any) as the corresponding bytecode behavior.

If the method referenced by R has the `ACC_VARARGS` flag set (§4.6), then the `java.lang.invoke.MethodHandle` instance is a variable arity method handle; otherwise, it is a fixed arity method handle.

A variable arity method handle performs argument list boxing (JLS §15.12.4.2) when invoked via `invoke`, while its behavior with respect to `invokeExact` is as if the `ACC_VARARGS` flag were not set.

Method handle resolution throws an `IncompatibleClassChangeError` if the method referenced by R has the `ACC_VARARGS` flag set and either A* is an empty sequence or the last parameter type in A* is not an array type. That is, creation of a variable arity method handle fails.

An implementation of the Java Virtual Machine is not required to intern method types or method handles. That is, two distinct symbolic references to method types or method handles which are structurally identical might not resolve to the same instance of `java.lang.invoke.MethodType` or `java.lang.invoke.MethodHandle` respectively.

The `java.lang.invoke.MethodHandles` class in the Java SE Platform API allows creation of method handles with no bytecode behavior. Their behavior is defined by the method of `java.lang.invoke.MethodHandles` that creates them. For example, a method handle may, when invoked, first apply transformations to its argument values, then supply the transformed values to the invocation of another method handle, then apply a transformation to the value returned from that invocation, then return the transformed value as its own result.

5.4.3.6. Dynamically-Computed Constant and Call Site Resolution

To resolve an unresolved symbolic reference R to a dynamically-computed constant or call site, there are three tasks. First, R is examined to determine which code will serve as its *bootstrap method*, and which arguments will be passed to that code. Second, the arguments are packaged into an array and the bootstrap method is invoked. Third, the result of the bootstrap method is validated, and used as the result of resolution.

The first task involves the following steps:

1. R gives a symbolic reference to a *bootstrap method handle*. The bootstrap method handle is resolved (§5.4.3.5) to obtain a `reference` to an instance of `java.lang.invoke.MethodHandle`.

   Any exception that can be thrown as a result of failure of resolution of a symbolic reference to a method handle can be thrown in this step.

   If R is a symbolic reference to a dynamically-computed constant, then let D be the type descriptor of the bootstrap method handle. (That is, D is a `reference` to an instance of `java.lang.invoke.MethodType`.) The first parameter type indicated by D must be `java.lang.invoke.MethodHandles.Lookup`, or else resolution fails with a `BootstrapMethodError`. For historical reasons, the bootstrap method handle for a dynamically-computed call site is not similarly constrained.

2. If R is a symbolic reference to a dynamically-computed constant, then it gives a field descriptor.

   If the field descriptor indicates a primitive type, then a `reference` to the pre-defined `Class` object representing that type is obtained (see the method `isPrimitive` in class `Class`).

   Otherwise, the field descriptor indicates a class or interface type, or an array type. A `reference` to the `Class` object representing the type indicated by the field descriptor is obtained, as if by resolution of an unresolved symbolic reference to a class or interface (§5.4.3.1) whose name corresponds to the type indicated by the field descriptor.

   Any exception that can be thrown as a result of failure of resolution of a symbolic reference to a class or interface can be thrown in this step.

3. If R is a symbolic reference to a dynamically-computed call site, then it gives a method descriptor.

   A `reference` to an instance of `java.lang.invoke.MethodType` is obtained, as if by resolution of an unresolved symbolic reference to a method type (§5.4.3.5) with the same parameter and return types as the method descriptor.

   Any exception that can be thrown as a result of failure of resolution of a symbolic reference to a method type can be thrown in this step.

4. R gives zero or more *static arguments*, which communicate application-specific metadata to the bootstrap method. Each static argument A is resolved, in the order given by R, as follows:

   - If A is a string constant, then a `reference` to its instance of class `String` is obtained.

- If A is a numeric constant, then a `reference` to an object representing the number is obtained by the following procedure:

  a. Let `v` be the value of the numeric constant, and let T be a field descriptor which corresponds to the type of the numeric constant.

  b. Let `MH` be a method handle produced as if by invocation of the `identity` method of `java.lang.invoke.MethodHandles` with an argument representing the class `Object`.

  c. A `reference` to an object is obtained as if by the invocation `MH.invoke(v)` with method descriptor `(T)Ljava/lang/Object;`.

- If A is a symbolic reference to a dynamically-computed constant with a field descriptor indicating a primitive type T, then A is resolved, producing a primitive value `v`. Given `v` and T, a `reference` is obtained to an object encoding `v` according to the procedure specified above for numeric constants.

- If A is any other kind of symbolic reference, then the result is the result of resolving A.

Among the symbolic references in the run-time constant pool, symbolic references to dynamically-computed constants are special because they are derived from `constant_pool` entries that can syntactically refer to themselves via the `BootstrapMethods` attribute (§4.7.23). However, the Java Virtual Machine does not support resolving a symbolic reference to a dynamically-computed constant that depends on itself (that is, as a static argument to its own bootstrap method). Accordingly, when both R and A are symbolic references to dynamically-computed constants, if A is the same as R or A gives a static argument that (directly or indirectly) references R, then resolution fails with a `StackOverflowError` at the point where re-resolution of R would be required.

Unlike class initialization (§5.5), where cycles are allowed between uninitialized classes, resolution does not allow cycles in symbolic references to dynamically-computed constants. If an implementation of resolution makes recursive use of a stack, then a `StackOverflowError` will occur naturally. If not, the implementation is required to detect the cycle rather than, say, looping infinitely or returning a default value for the dynamically-computed constant.

A similar cycle may arise if the body of a bootstrap method makes reference to a dynamically-computed constant currently being resolved. This has always been possible for invokedynamic bootstraps, and does not require special treatment in resolution; the recursive `invokeWithArguments` calls will naturally lead to a `StackOverflowError`.

Any exception that can be thrown as a result of failure of resolution of a symbolic reference can be thrown in this step.

The second task, to invoke the bootstrap method handle, involves the following steps:

1. An array is allocated with component type `Object` and length *n*+3, where *n* is the number of static arguments given by R (*n* ≥ 0).

   The zeroth component of the array is set to a `reference` to an instance of `java.lang.invoke.MethodHandles.Lookup` for the class in which R occurs, produced as if by invocation of the `lookup` method of `java.lang.invoke.MethodHandles`.

   The first component of the array is set to a `reference` to an instance of `String` that denotes `N`, the unqualified name given by R.

   The second component of the array is set to the `reference` to an instance of `Class` or `java.lang.invoke.MethodType` that was obtained earlier for the field descriptor or method descriptor given by R.

   Subsequent components of the array are set to the `reference`s that were obtained earlier from resolving R's static arguments, if any. The `reference`s appear in the array in the same order as the corresponding static arguments are given by R.

   A Java Virtual Machine implementation may be able to skip allocation of the array and, without any change in observable behavior, pass the arguments directly to the bootstrap method.

2. The bootstrap method handle is invoked, as if by the invocation `BMH.invokeWithArguments(args)`, where `BMH` is the bootstrap method handle and `args` is the array allocated above.

   Due to the behavior of the `invokeWithArguments` method of `java.lang.invoke.MethodHandle`, the type descriptor of the bootstrap method handle need not exactly match the run-time types of the arguments. For example, the second parameter type of the bootstrap method handle (corresponding to the unqualified name given in the first component of the array above) could be `Object` instead of `String`. If the bootstrap method handle is variable arity, then some or all of the arguments may be collected into a trailing array parameter.

   The invocation occurs within a thread that is attempting resolution of this symbolic reference. If there are several such threads, the bootstrap method handle may be invoked concurrently. Bootstrap methods which access global application data should take the usual precautions against race conditions.

   If the invocation fails by throwing an instance of `Error` or a subclass of `Error`, resolution fails with that exception.

   If the invocation fails by throwing an exception that is not an instance of `Error` or a subclass of `Error`, resolution fails with a `BootstrapMethodError` whose cause is the thrown exception.

   If several threads concurrently invoke the bootstrap method handle for this symbolic reference, the Java Virtual Machine chooses the result of one invocation and installs it visibly to all threads. Any other bootstrap methods executing for this symbolic reference are allowed to complete, but their results are ignored.

The third task, to validate the `reference`, `o`, produced by invocation of the bootstrap method handle, is as follows:

- If R is a symbolic reference to a dynamically-computed constant, then `o` is converted to type T, the type indicated by the field descriptor given by R.

  `o`'s conversion occurs as if by the invocation `MH.invoke(o)` with method descriptor `(Ljava/lang/Object;)T`, where `MH` is a method handle produced as if by invocation of the `identity` method of `java.lang.invoke.MethodHandles` with an argument representing the class `Object`.

  The result of `o`'s conversion is the result of resolution.

  If the conversion fails by throwing a `NullPointerException` or a `ClassCastException`, resolution fails with a `BootstrapMethodError`.

- If R is a symbolic reference to a dynamically-computed call site, then `o` is the result of resolution if it has all of the following properties:

  - `o` is not `null`.
  - `o` is an instance of `java.lang.invoke.CallSite` or a subclass of `java.lang.invoke.CallSite`.
  - The type of the `java.lang.invoke.CallSite` is semantically equal to the method descriptor given by R.

  If `o` does not have these properties, resolution fails with a `BootstrapMethodError`.

Many of the steps above perform computations "as if by invocation" of certain methods. In each case, the invocation behavior is given in detail by the specifications for *invokestatic* and *invokevirtual*. The invocation occurs in the thread and from the class that is attempting

resolution of the symbolic reference R. However, no corresponding method references are required to appear in the run-time constant pool, no particular method's operand stack is necessarily used, and the value of the `max_stack` item of any method's `Code` attribute is not enforced for the invocation.

5.4.4. Access Control

Access control is applied during resolution (§5.4.3) to ensure that a reference to a class, interface, field, or method is permitted. Access control succeeds if a specified class, interface, field, or method is *accessible* to the referring class or interface.

A class or interface C is accessible to a class or interface D if and only if one of the following is true:

- C is `public`, and a member of the same run-time module as D (§5.3.6).
- C is `public`, and a member of a different run-time module than D, and C's run-time module is read by D's run-time module, and C's run-time module exports C's run-time package to D's run-time module.
- C is not `public`, and C and D are members of the same run-time package.

If C is not accessible to D, then access control throws an `IllegalAccessError`. Otherwise, access control succeeds.

A field or method R is accessible to a class or interface D if and only if any of the following is true:

- R is `public`.
- R is `protected` and is declared in a class C, and D is either a subclass of C or C itself.

  Furthermore, if R is not `static`, then the symbolic reference to R must contain a symbolic reference to a class T, such that T is either a subclass of D, a superclass of D, or D itself.

  During verification of D, it was required that, even if T is a superclass of D, the target reference of a `protected` field access or method invocation must be an instance of D or a subclass of D (§4.10.1.8).

- R is either `protected` or has default access (that is, neither `public` nor `protected` nor `private`), and is declared by a class in the same run-time package as D.
- R is `private` and is declared by a class or interface C that belongs to the same nest as D, according to the nestmate test below.

If R is not accessible to D, then access control throws an `IllegalAccessError`. Otherwise, access control succeeds.

A *nest* is a set of classes and interfaces that allow mutual access to their `private` members. One of the classes or interfaces is the *nest host*. It enumerates the classes and interfaces which belong to the nest, using the `NestMembers` attribute (§4.7.29). Each of them in turn designates it as the nest host, using the `NestHost` attribute (§4.7.28). A class or interface which lacks a `NestHost` attribute belongs to the nest hosted by itself; if it also lacks a `NestMembers` attribute, then this nest is a singleton consisting only of the class or interface itself.

The Java Virtual Machine determines the nest to which a given class or interface belongs (that is, the nest host designated by the class or interface) as part of access control, rather than when the class or interface is loaded. Certain methods of the Java SE Platform API may determine the nest to which a given class or interface belongs prior to access control, in which case the Java Virtual Machine respects that prior determination during access control.

To determine whether a class or interface C belongs to the same nest as a class or interface D, the *nestmate test* is applied. C and D belong to the same nest if and only if the nestmate test succeeds. The nestmate test is as follows:

- If C and D are the same class or interface, then the nestmate test succeeds.
- Otherwise, the following steps are performed, in order:
  1. Let H be the nest host of D, if the nest host of D has previously been determined. If the nest host of D has *not* previously been determined, then it is determined using the algorithm below, yielding H.
  2. Let H' be the nest host of C, if the nest host of C has previously been determined. If the nest host of C has *not* previously been determined, then it is determined using the algorithm below, yielding H'.
  3. H and H' are compared. If H and H' are the same class or interface, then the nestmate test succeeds. Otherwise, the nestmate test fails.

The nest host of a class or interface `M` is determined as follows:

- If `M` lacks a `NestHost` attribute, then `M` is its own nest host.
- Otherwise, `M` has a `NestHost` attribute, and its `host_class_index` item is used as an index into the run-time constant pool of `M`. The symbolic reference at that index is resolved (§5.4.3.1).

  If resolution of the symbolic reference fails, then `M` is its own nest host. Any exception thrown as a result of failure of class or interface resolution is *not* rethrown.

  Otherwise, resolution of the symbolic reference succeeds. Let H be the resolved class or interface. The nest host of `M` is determined by the following rules:

  - If any of the following is true, then `M` is its own nest host:
    - H is not in the same run-time package as `M`.
    - H lacks a `NestMembers` attribute.
    - H has a `NestMembers` attribute, but there is no entry in its `classes` array that refers to a class or interface with the name `N`, where `N` is the name of `M`.
  - Otherwise, H is the nest host of `M`.

5.4.5. Method Overriding

An instance method m_C *can override* another instance method m_A iff all of the following are true:

- m_C has the same name and descriptor as m_A.

- m_C is not marked `ACC_PRIVATE`.
- One of the following is true:
  - m_A is marked `ACC_PUBLIC`.
  - m_A is marked `ACC_PROTECTED`.
  - m_A is marked neither `ACC_PUBLIC` nor `ACC_PROTECTED` nor `ACC_PRIVATE`, and either (a) the declaration of m_A appears in the same run-time package as the declaration of m_C, or (b) if m_A is declared in a class A and m_C is declared in a class C, then there exists a method m_B declared in a class B such that C is a subclass of B and B is a subclass of A and m_C can override m_B and m_B can override m_A.

Part (b) of the final case allows for "transitive overriding" of methods with default access. For example, given the following class declarations in a package P:

```
public class A           {        void m() {} }
public class B extends A { public void m() {} }
public class C extends B {        void m() {} }
```

and the following class declaration in a different package:

```
public class D extends P.C { void m() {} }
```

then:

- *`B.m` can override `A.m`.*
- *`C.m` can override `B.m` and `A.m`.*
- *`D.m` can override `B.m` and, transitively, `A.m`, but it cannot override `C.m`.*

5.4.6. Method Selection

During execution of an *invokeinterface* or *invokevirtual* instruction, a method is *selected* with respect to (i) the run-time type of the object on the stack, and (ii) a method that was previously *resolved* by the instruction. The rules to select a method with respect to a class or interface C and a method m_R are as follows:

1. If m_R is marked `ACC_PRIVATE`, then it is the selected method.
2. Otherwise, the selected method is determined by the following lookup procedure:
   - If C contains a declaration of an instance method `m` that can override m_R (§5.4.5), then `m` is the selected method.
   - Otherwise, if C has a superclass, a search for a declaration of an instance method that can override m_R is performed, starting with the direct superclass of C and continuing with the direct superclass of that class, and so forth, until a method is found or no further superclasses exist. If a method is found, it is the selected method.
   - Otherwise, the maximally-specific superinterface methods of C are determined (§5.4.3.3). If exactly one matches m_R's name and descriptor and is not `abstract`, then it is the selected method.

     Any maximally-specific superinterface method selected in this step can override m_R; there is no need to check this explicitly.

While C will typically be a class, it may be an interface when these rules are applied during preparation (§5.4.2).

5.5. Initialization

Initialization of a class or interface involves assigning any `ConstantValue` attribute values to its `static` fields and executing any declared class or interface initialization method (§2.9.2).

A class or interface C may be initialized only as a result of:

- The execution of any one of the Java Virtual Machine instructions *new*, *getstatic*, *putstatic*, or *invokestatic* that references C (§*new*, §*getstatic*, §*putstatic*, §*invokestatic*).

  Upon execution of a *new* instruction, the class to be initialized is the class referenced by the instruction.

  Upon execution of a *getstatic*, *putstatic*, or *invokestatic* instruction, the class or interface to be initialized is the class or interface that declares the resolved field or method.
- The first invocation of a `java.lang.invoke.MethodHandle` instance which was the result of method handle resolution (§5.4.3.5) for a method handle of kind 2 (`REF_getStatic`), 4 (`REF_putStatic`), 6 (`REF_invokeStatic`), or 8 (`REF_newInvokeSpecial`).

  This implies that the class of a bootstrap method is initialized when the bootstrap method is invoked for an invokedynamic instruction (§invokedynamic), as part of the continuing resolution of the call site specifier.
- Invocation of certain reflective methods in the class library (§2.12), for example, in class `Class` or in package `java.lang.reflect`.
- If C is a class, the initialization of one of its subclasses.
- If C is an interface that declares a non-`abstract`, non-`static` method, the initialization of a class that implements C directly or indirectly.
- Its designation as the initial class or interface at Java Virtual Machine startup (§5.2).

Prior to initialization, a class or interface must be linked, that is, verified, prepared, and optionally resolved.

Because the Java Virtual Machine is multithreaded, initialization of a class or interface requires careful synchronization, since some other thread may be trying to initialize the same class or interface at the same time. There is also the possibility that initialization of a class or interface may be requested recursively as part of the initialization of that class or interface. The implementation of the Java Virtual Machine is responsible for taking care of synchronization and recursive initialization by using the following procedure. It assumes that the class or interface has already been verified and prepared, and that the class or interface contains state that indicates one of four situations:

- This class or interface is verified and prepared but not initialized.
- This class or interface is being initialized by some particular thread.
- This class or interface is fully initialized and ready for use.
- This class or interface is in an erroneous state, perhaps because initialization was attempted and failed.

The precise form of the initialization state is left to the discretion of the JVM implementation.

For each class or interface C, there is a unique initialization lock `LC`. The mapping from C to `LC` is also left to the discretion of the Java Virtual Machine implementation. For example, `LC` could be the `Class` object for C, or the monitor associated with that `Class` object. The procedure for initializing C is then as follows:

1. Synchronize on the initialization lock, `LC`, for C. This involves waiting until the current thread can acquire `LC`.
2. If the initialization state of C indicates that initialization is in progress for C by some other thread, then release `LC` and block the current thread until informed that the in-progress initialization has completed, at which time repeat this procedure.

   Thread interrupt status is unaffected by execution of the initialization procedure.
3. If the initialization state of C indicates that initialization is in progress for C by the current thread, then this must be a recursive request for initialization. Release `LC` and complete normally.
4. If the initialization state of C indicates that C has already been initialized, then no further action is required. Release `LC` and complete normally.
5. **If the initialization state of C is in an erroneous state, then initialization is not possible. Release `LC` and throw a `NoClassDefFoundError`.**
6. Otherwise, record the fact that initialization of C is in progress by the current thread, and release `LC`.

   Then, initialize each `static` field of C with the constant value in its `ConstantValue` attribute (§4.7.2), in the order the fields appear in the `ClassFile` structure.
7. Next, if C is a class rather than an interface, then let SC be its superclass and let SI_1, ..., SI_n be all superinterfaces of C (whether direct or indirect) that declare at least one non-`abstract`, non-`static` method. The order of superinterfaces is given by a recursive enumeration over the superinterface hierarchy of each interface directly implemented by C. For each interface I directly implemented by C (in the order of the `interfaces` array of C), the enumeration recurs on I's superinterfaces (in the order of the `interfaces` array of I) before returning I.

   For each S in the list [SC, SI_1, ..., SI_n], if S has not yet been initialized, then recursively perform this entire procedure for S. If necessary, verify and prepare S first.

   If the initialization of S completes abruptly because of a thrown exception, then acquire `LC`, label C as erroneous, notify all waiting threads, release `LC`, and complete abruptly, throwing the same exception that resulted from initializing S.
8. Next, determine whether assertions are enabled for C by querying its defining loader.
9. Next, if C declares a class or interface initialization method, execute that method.
10. If the execution of the class or interface initialization method completes normally, or if C declares no class or interface initialization method, then acquire `LC`, label C as fully initialized, notify all waiting threads, release `LC`, and complete this procedure normally.
11. **Otherwise, the class or interface initialization method must have completed abruptly by throwing some exception E. If the class of E is not `Error` or one of its subclasses, then create a new instance of the class `ExceptionInInitializerError` with E as the argument, and use this object in place of E in the following step. If a new instance of `ExceptionInInitializerError` cannot be created because an `OutOfMemoryError` occurs, then use an `OutOfMemoryError` object in place of E in the following step.**
12. **Acquire `LC`, label C as erroneous, notify all waiting threads, release `LC`, and complete this procedure abruptly with reason E or its replacement as determined in the previous step.**

A Java Virtual Machine implementation may optimize this procedure by eliding the lock acquisition in step 1 (and release in step 4/5) when it can determine that the initialization of the class has already completed, provided that, in terms of the Java memory model, all *happens-before* orderings (JLS §17.4.5) that would exist if the lock were acquired, still exist when the optimization is performed.

5.6. Binding Native Method Implementations

Binding is the process by which a function written in a language other than the Java programming language and implementing a `native` method is integrated into the Java Virtual Machine so that it can be executed. Although this process is traditionally referred to as linking, the term binding is used in the specification to avoid confusion with linking of classes or interfaces by the Java Virtual Machine.

5.7. Java Virtual Machine Termination

The Java Virtual Machine executes code in threads (§2.5). A thread is either a non-daemon thread, a daemon thread, or a shutdown hook.

Readers are referred to the API specifications of `Thread` and `Runtime` for details of how threads obtain daemon status, and how shutdown hooks are registered.

A thread *terminates* if either (i) its `run` method completes normally, or (ii) its `run` method completes abruptly and the relevant uncaught exception handler (§2.10) completes normally or abruptly. With no code left to run, the thread has completed execution and therefore has no current method (§2.5.1).

The Java Virtual Machine *terminates* when one of the following situations has occurred:

1. A thread invoked `System.exit` or `Runtime.exit`, and all of the shutdown hooks which consequently were started by the Java Virtual Machine, if any, have terminated.
2. A thread invoked `Runtime.halt`. (No shutdown hooks are started in this situation.)
3. The Java Virtual Machine implementation recognized an external event as requesting termination of the Java Virtual Machine, and all of the shutdown hooks which consequently were started by the Java Virtual Machine, if any, have terminated.

   The nature of the event is outside the scope of this specification, but is necessarily something that a Java Virtual Machine implementation can handle reliably. An example is receiving a signal from the operating system.
4. An external event occurred that the Java Virtual Machine implementation cannot handle. (No shutdown hooks are started in this situation.)

   The nature of the event is outside the scope of this specification, but is necessarily something that a Java Virtual Machine implementation cannot recognize or recover from in any way. Examples include a fatal error occurring in the process running the implementation, or power being removed from the computer running the implementation.

Upon Java Virtual Machine termination, any daemon or non-daemon thread that has not yet terminated will execute no further Java code. The current method of the thread does not complete normally or abruptly.

If the Java Virtual Machine terminates because a thread invoked `Runtime.halt` *while shutdown hooks were running*, then, in addition to daemon and non-daemon threads, any shutdown hook that has not yet terminated will execute no further Java code.

Native applications can use the JNI Invocation API to create and destroy the Java Virtual Machine in such a way that a Java program, having started execution in the `main` method of an initial class (JLS §12.1), exits when all of its non-daemon threads have terminated (JLS §12.8). The Java Virtual Machine does not terminate "automatically" when the last non-daemon thread terminates.

Legal Notice